
One ROM

One ROM CLI Manual

The command-line tool for managing One ROMs.

One ROM CLI v0.4.0

8 September 2026

Built from one-rom ea91580 · source `docs/CLI-MANUAL.md`

Copyright © 2026 Piers Finlayson <piers@piers.rocks> · MIT licence

<https://onerom.org>

Contents

- Introduction5
 - About This Document 5
- New Breaking Changes 6
- One ROM Overview7
 - Background - ROM Replacements7
 - Introduction to One ROM7
 - Hardware.....7
 - Important concepts..... 8
 - Types of One ROM Deployments 9
 - Minimal 9
 - Standard 9
 - Extended 9
 - Custom10
- CLI Guide 11
 - Installation11
 - Keeping the CLI up to date11
 - How One ROM talks to the CLI 12
 - Identifying your device 12
 - Common workflows 13
 - Program a device from a config file 13
 - Program from `--slot` specifications 13
 - Program with a plugin 13
 - Plugin compatibility..... 13
 - Build firmware without flashing 14
 - Inspect a device 14
 - Read the live ROM image..... 14
 - Patch a running image 14
 - Identify a physical unit 15
 - Reset the host system after programming 15
 - See what One ROM is doing with its GPIOs 15
 - Watch a device's log..... 15
 - Erase a device..... 16
 - Prepare a 16-bit ROM image..... 16
 - Device states 16
 - Global behaviour worth knowing..... 17
- CLI Reference 18
 - Synopsis..... 18

Global options	18
Command summary.....	18
scan.....	20
program.....	21
Source of the firmware (mutually exclusive groups)	21
Configuration metadata	21
Per-device overrides	22
Board, version and output	22
Reboot and flashing behaviour.....	22
inspect.....	24
inspect info.....	24
inspect telemetry	24
inspect slots	24
inspect image.....	24
inspect peek.....	25
inspect gpio.....	25
inspect led.....	28
inspect rgb	29
inspect header	30
inspect socket.....	30
monitor	31
monitor log	31
control	33
control reboot.....	33
control led	33
control rgb.....	34
control poke	36
control reset	37
control select	38
control pin	38
control erase	39
update	41
update slot	41
update commit	41
update otp.....	41
image.....	42
image swap-bytes	42
image deinterleave.....	42
image convert	43
firmware	45
firmware build	45

firmware inspect	46
firmware releases	46
firmware download	46
firmware chips	47
plugin.....	48
chips.....	49
Flash usage	49
board	51
board list.....	51
board header.....	51
board socket	52
self.....	54
self check.....	54
self download.....	54
Top-level aliases	56
peek (top-level alias)	56
poke (top-level alias)	56
reboot (top-level alias).....	56
ROM slot specification.....	57
Image transforms.....	59
16-bit ROM image byte ordering.....	60
Plugin specification.....	62
Pin values.....	63
Problems	64
Recovering a bricked One ROM	64
Situation	64
Bootling into the bootloader	64
Checking the host can see it	65
Programming it again.....	65
Ice boards	65
Appendix: Breaking Change History	66
v0.4.0	66
v0.3.0	66

Introduction

`onerom` (`onerom.exe` on Windows) is the command-line tool for managing One ROM ROM emulators: discovering connected devices, building and flashing firmware, inspecting device state, and manipulating ROM image files.

About This Document

This One ROM CLI manual covers:

- **One ROM Overview** — what One ROM is, its hardware, and the vocabulary the rest of this manual uses.
- **CLI Guide** — installation and the common workflows.
- **CLI Reference** — every command, subcommand and option.
- **Problems** — symptoms and their fixes, including [recovering a bricked One ROM](#).

This manual documents the `onerom` CLI as of release v0.4.0. Board, chip and plugin lists shown in examples are illustrative — the set your build supports may differ. Run `onerom --version` to check your version, and `onerom board list` / `onerom chips` for the definitive lists your build knows about. Commands marked **(not yet supported)** are present in the CLI surface but not yet functional.

New Breaking Changes

These change what an existing command line does, so a script written against an earlier release is worth checking before upgrading.

The CLI is versioned `MAJOR.MINOR.PATCH`. While it is pre-v1.0.0, a change that breaks an existing command line lands in a minor release — v0.3.0 to v0.4.0 — and never in a patch. From v1.0.0 onwards such a change lands in a major release, and never in a minor or a patch.

- **`--name` now names the One ROM, not the configuration.** It is an alias for `--instance-name` on `program` and `firmware build`, where it was an alias for `--config-name`. A command line using `--name` still runs, and names the device instead of the configuration it is building. Spell `--config-name` in full for the old meaning.

Every release's breaking changes are collected in [Appendix: Breaking Change History](#), at the end of this manual.

One ROM Overview

Background - ROM Replacements

ROM (Read-Only Memory) chips are used in a huge assortment of electronic devices to supply those systems with pre-programmed data - Operating System and BIOS images, character sets, programming languages, games, etc. On old systems these sometimes fail and need replacing, or users would like to upgrade their system, by replacing with a ROM with newer data. The original ROM chip is removed from the system and a replacement is installed in its place.

Traditional ROM replacements are built around EPROMs, EEPROMs and flash chips. Those are similar devices to the ROM being replaced - that is dumb devices that have a persistent memory store and some control lines - but with higher capacity than the original ROM and/or different chip select behaviour. The chip select logic is typically fixed with additional on-board logic.

Introduction to One ROM

One ROM takes a different approach - it consists of a microcontroller paired with flash and RAM, physically laid out on a PCB so that the original ROM's address, data and chip select lines connect to the MCU's general-purpose input/output (GPIO) pins. This makes One ROM a ROM emulator, also known as a Software Defined ROM.

Firmware is loaded to One ROM's flash, along with the ROM image(s) to be served. One ROM is installed in the system. On power on, it boots and first loads and runs its firmware, and then loads the ROM image(s) to be served into RAM, and serves them from there.

Because One ROM is microcontroller based, it is controlled by software rather than by hardwired, hardware-based logic. This makes One ROM the most powerful and flexible ROM replacement available. Nearly any original ROM of a particular form factor can be replaced by an equivalent One ROM.

One ROM's software is open source and has a modular architecture with a rich plugin API. This means One ROM is a platform that can be extended with new capabilities, not just a ROM replacement. Some examples of ways in which people have extended One ROM include:

- A comprehensive USB stack and interface to allow One ROM to be managed while running, built on the public plugin API.
- Communication between a retro system and a PC using One ROM's USB port.
- A retro system controlling its own reset line using one of One ROM's header pins.
- Reprogramming One ROM dynamically as part of a car's ECU tuning process.

Hardware

One ROM comes in multiple physical variants to support each supported ROM package size - 24, 28, 32 and 40 pin. All One ROMs replace original 600 mil (15.24mm) wide DIP packages. The supported ROM types are listed in [COMPATIBILITY](#). Each pin variant has multiple hardware revisions, marked on the board.

One ROM Fire, based on the RP2350 microcontroller, is the current family and is available in all four sizes.

One ROM Ice is a deprecated 24 pin only family based on an STM32F4. It is supported on Base Firmware v0.6.x. New feature releases for One ROM Ice are not anticipated.

Important concepts

- **Base Firmware** - a binary including the core code that causes a One ROM to serve ROM images. Does not include any information about the One ROM hardware variant or the ROM images to serve. Should not be used on its own to program a One ROM device.
- **ROM image** - a binary blob that is the contents of a ROM to be served. Can be supplied to the programming tool as a raw binary file, or in other formats, including Intel HEX and Motorola S-record. The programming tool converts the ROM image to the format One ROM requires.
- **Slot** - a region of One ROM's flash holding a single ROM image, set of ROM images, or plugin.
- **Image selection** - the choice of which ROM image a One ROM serves. A One ROM can hold multiple images, selected at boot using the image select jumpers on the board.
- **Configuration** - the description of what a One ROM should do, provided to the programmer. It names the ROM images to serve, the chip type and chip select behaviour of each, and any settings.
- **Programmer** - a tool that composes ROM images, configuration, Base Firmware and metadata into a complete image, and flashes it to the device. There are three - the [web programmer](#), the [CLI](#) and [Studio](#).
- **Metadata** - a binary block that describes the hardware properties of a physical One ROM device, and the properties of the ROM images to be served. The programmer generates this metadata from the configuration provided, and it is included in the firmware that is flashed to the device.
- **Bootloader** - a small program contained in One ROM's own read-only memory that allows One ROM to be reflashed with new firmware and metadata. The bootloader is always accessible, even if the firmware and metadata are corrupted or missing and, because it is ROM based itself, it cannot be corrupted or erased.
- **Header pins** - pins on the One ROM board exposing power, ground, the image select lines, the debug interface and spare pins for expansion. They allow One ROM to be wired to the system it is installed in, or to external hardware.
- **Plugin** - a binary blob consisting of code that extends One ROM's capabilities beyond those provided by the core firmware. The most common plugin is the System Plugin. A user chooses which plugins to include in their firmware (if any) at programming time. Up to two plugins can be included with a single One ROM firmware image - the System Plugin, and a User Plugin.
- **System Plugin** - a plugin that provides a USB stack and other capabilities to One ROM. One ROM's System Plugin is shipped alongside One ROM's Base Firmware. Its use is recommended for all users, except those who want to replace it with their own custom, replacement plugin.
- **User Plugin** - a plugin installed alongside the System Plugin providing additional capabilities to One ROM. User Plugins can be developed by anyone, and a number of them are shipped alongside Base Firmware and System Plugin. User Plugins require a System Plugin to be included. An example User Plugin is One

ROM's Host Control plugin, which allows One ROM to be controlled by the system it is installed in, with no extra wiring using the ROM Bus Control Protocol (RBCP).

- **Setting** - a named value that changes how a One ROM behaves rather than what it serves. Settings are configured by the user and written by the programmer. The status LED and the CPU frequency are example settings supported by the Base Firmware. Plugins may have their own settings.
- **ROM Bus Control Protocol (RBCP)** - a **protocol** that allows ROM emulators like One ROM to be controlled by a system it is installed in, with no extra wiring. Supported by One ROM's Host Control plugin.
- **Bricked One ROM** - a One ROM device that is non-functional because it has been flashed with corrupted firmware or metadata, metadata for a different physical One ROM device, no metadata, or firmware or metadata with some other issue. Bricked One ROMs can be recovered by entering One ROM's bootloader and reflashing them with valid firmware and metadata. It is very unlikely that a bricked One ROM cannot be recovered.

Types of One ROM Deployments

Minimal

Consists of:

- Base Firmware
- Metadata
- At least one ROM image

Contains no plugins. Serves the ROM image(s) to the system it is installed in, but cannot be managed while running.

In this deployment type, when One ROM is plugged in via USB it drops automatically into its bootloader, allowing the device to be reprogrammed, and simultaneously stops serving the ROM image(s) to the system it is installed in.

Standard

A Minimal Deployment plus One ROM's System Plugin, which includes a USB stack enabling comprehensive management of the device while it is running.

This is the recommended deployment type for most users.

Extended

A Standard Deployment plus a User Plugin. The User Plugin may be One ROM's own, or one written by a third party.

Custom

A Custom Deployment is one where one or more of the following holds:

- The physical One ROM device is not manufactured from a **published design**. It may be a derivative, or a fully custom design.
- The Base Firmware is a fork of or replacement for One ROM's Base Firmware.
- The System Plugin is replaced with a fork of or replacement for One ROM's System Plugin.

CLI Guide

Installation

Download the CLI from <https://onerom.org/cli>. Builds are provided for:

- Windows — x86 64-bit and ARM 64-bit
- macOS - a single universal build for Intel and Apple Silicon
- Ubuntu/Debian — x86 64-bit, and ARM 64-bit (also for Raspberry Pi)

The Windows and macOS builds are digitally signed. A sha256 checksum is published alongside every download so you can verify what you fetched.

Windows / macOS — unzip the archive and place the `onerom` (`onerom.exe`) executable in a folder on your `PATH`.

Linux — install the `.deb` as usual, which places `onerom` on your `PATH`:

```
sudo dpkg -i onerom-cli-x.y.z-1_amd64.deb
```

(replace `x.y.z` with the version, e.g. `0.1.10`, and `amd64` with `arm64` for the ARM/Pi build).

Windows SmartScreen — as a relatively new publisher, the first run may raise a *"Windows protected your PC"* dialog. Click **More info**, confirm the publisher reads *"Open Source Developer, Piers Finlayson"*, then **Run anyway**.

Verify it runs:

```
onerom --version
```

Keeping the CLI up to date

The CLI does not check for updates on its own. `onerom self check` compares the current build against the newest release published for your platform, and `onerom self download` fetches it:

```
onerom self check
onerom self download
```

`self download` saves the same artifact you would get from <https://onerom.org/cli> — a `.deb` on Linux, a zip on Windows and macOS — verifies it against the SHA-256 published alongside it, and prints the install step for what it downloaded. You must install the new version.

How One ROM talks to the CLI

The CLI communicates with a One ROM over USB using picoboot (the Raspberry Pi bootloader protocol, extended by [picobootx](#)). A One ROM is reachable in two situations:

- **Running** — normal firmware is running and serving ROMs; its USB stack (provided by the system USB plugin) exposes the picobootx interface.
- **Stopped** — the device is in One ROM's bootloader (BOOTSEL). A bare RP2350 bootloader is also reachable here, which is how unprogrammed or bricked units are [recovered](#).

Some commands work in either state; some require one specifically. Each reference entry notes when a device connection is required, and the state model is summarised under [Device states](#).

Identifying your device

With **exactly one** One ROM connected that the CLI recognises, you don't need to identify it — commands find it automatically, and the board type is inferred from the device.

With **multiple** devices connected, select one with `--serial` (`-s`). It accepts `*` and `?` wildcards:

```
onerom --serial 'A1B2*' inspect info
```

`--serial` is **global**: it can appear at any level of the command line.

A device programmed with a serial override (`program --serial-override`) reports and is matched by that overridden serial while **Running**. When **Stopped** its USB stack comes from the RP2350 bootrom, so it falls back to reporting its chip ID — match it by chip ID (or reboot it to running) in that state.

Discover what's attached:

```
onerom scan
onerom scan --slots      # also list each device's ROM slots
```

Two situations need extra flags:

- **Unrecognised / unprogrammed / bricked** units: add `--unrecognised` (`-u`) and supply `--board`, since the board type can't be inferred. The unit must still answer on its picoboot USB interface — one that answers nothing is ignored either way, since it cannot be programmed. It reports no serial at all, so `--serial` cannot pick between two of them — attach one at a time, and see [Recovering a bricked One ROM](#).
- **Non-standard USB IDs**: add `--vid-pid <VID:PID>` (hex), repeatable. When supplied, only the given VID/PID pairs are matched.

`--board` (`-b`) can also be given on most commands to **override** the detected board type.

Common workflows

Program a device from a config file

The primary workflow. Build firmware from a JSON config and flash it in one step:

```
onerom program --config c64.json
```

`program` builds *and* flashes. To build a firmware binary **without** flashing, use `firmware build` instead. To build and also keep the binary while flashing, add `--output` :

```
onerom program --config c64.json --out firmware.bin
```

Program from `--slot` specifications

Instead of a config file, describe each ROM slot inline. Repeat `--slot` per slot. The required chip-select lines depend on the chip type (e.g. a 2332 needs `cs1` and `cs2`):

```
onerom program --board fire-24-e \
  --slot file=kernal.bin,type=2364,cs1=active-low \
  --slot file=basic.bin,type=2364,cs1=active-low
```

The full slot spec grammar is documented under [ROM slot specification](#) — it covers CS polarity, size handling, per-slot CPU frequency/voltage, the status LED and 16-bit forcing.

Program with a plugin

Plugins masquerade as ROMs. At most one system plugin and one user plugin are supported; a user plugin requires a system plugin. The system plugin lands in slot 0, the user plugin in slot 1:

```
onerom program --board fire-24-e \
  --slot file=kernal.bin,type=2364,cs1=active-low \
  --plugin usb
```

`--plugin` may also be combined with `--config`. The plugins are inserted ahead of the config's ROM slots (which shift up accordingly), so you can add a plugin to a stock config without editing it:

```
onerom program --config c64.json --plugin usb
```

It is an error if the config already defines a plugin of its own — remove it from the config, or drop `--plugin`.

Plugin spec forms are listed under [Plugin specification](#).

Plugin compatibility

Every plugin going into an image is checked against the compatibility window published on the images server, whether it arrived via `--plugin` or was named by the config's own slots. A plugin the target firmware falls outside the window of is refused, and no image is written or flashed:

```
$ onerom firmware build --config usb-0.1.2.json --board fire-24-a --output fw.bin
Failed to execute command.
Plugin 'usb' version '0.1.2' is not compatible with firmware 0.7.0 or later.
The selected firmware version is 0.7.1.
Plugin version 0.2.1 supports it: https://images.onerom.org/plugins/system/usb/v0.2.1/plugin.bin
```

The last line names the newest release that does support the firmware being built for, and the URL to point the config's plugin slot at. If no release of that plugin supports it, the message says so instead. A pinned `--plugin usb,version=0.1.2` is refused the same way, with the same suggestion.

The check is worth having because a plugin binary declares only the *minimum* firmware it needs. A release withdrawn for some *newer* firmware — One ROM USB v0.1.2, which hard faults on firmware v0.7.0 — is recorded only in the manifest, so this is the one place it can be caught before the device stops booting.

`--verbose` reports a plugin that passed:

```
Plugin 'usb' v0.2.1 is compatible with firmware
```

A plugin loaded from a local path, or from any URL that is not an official images-server one, has no published compatibility to check against and is built in as-is. Under `--verbose`:

```
Plugin /home/me/my-plugin.bin is not an official One ROM plugin - no published compatibility to check
```

If the images server cannot be reached the build is not blocked — an offline build of an otherwise valid config still works — but a `Warning:` line naming the plugin says the check was skipped.

Build firmware without flashing

```
onerom firmware build --config c64.json --board fire-24-e --out firmware.bin
```

Inspect a device

```
onerom inspect info      # serial, name, board, MCU, firmware version, hw revision
onerom inspect slots     # ROM slots, with the active one marked
```

Read the live ROM image

Read what the device would serve for a given logical ROM address (device must be running). The top-level `peek` is an alias for `inspect peek live`:

```
onerom peek --address 0x100 --length 64
onerom peek --address 0 --length 8192 --output rom-image.bin
```

Patch a running image

`poke live` writes to the ROM image currently being served, at a logical ROM offset. Changes are transient — lost on reboot. The top-level `poke` is an alias for `control poke live`:

```
onerom poke --address 0x100 --byte 0xEA
onerom poke --address 0 --input patch.bin
```

For file patches you can write only the differing bytes, and preview first:

```
onerom poke --input patch.bin --delta --dry-run
onerom poke --input patch.bin --delta
```

Identify a physical unit

Make the status LED beacon so you can spot which board is which:

```
onerom control led beacon
```

Reset the host system after programming

If you have run a wire from a One ROM header pad to the reset line of the machine One ROM is installed in, One ROM can pulse that pad low and then release it — resetting the host so it picks up the image you just flashed. Name the pad, or the MCU GPIO behind it — `onerom inspect header` shows which that is:

```
onerom program --config c64.json --reset-host sel_c
```

`--reset-host` waits for the One ROM to come back on the USB bus and then sends the pulse, so programming and resetting are one command. To reset a host without programming it, or to choose the length of the pulse, use `control reset`:

```
onerom control reset --pin sel_c
onerom control reset --pin sel_c --hold 500
```

The pad is typically an image-select pad whose jumper you have removed, usually `sel_c`, or an `X1 / X2` pad. The device times the pulse, so an interrupted CLI cannot leave the host held in reset. See `control reset`, and `control pin` for driving a GPIO to an arbitrary state.

See what One ROM is doing with its GPIOs

```
onerom inspect gpio
```

One row per MCU GPIO: everything that GPIO is — its ROM socket signal under the image being served, the board peripheral it drives, the header pad it surfaces on — plus direction, level, 5V tolerance, and what One ROM itself is using it for. GPIOs connected to nothing are omitted unless you pass `--all`. Useful before driving a pin — see `inspect gpio`.

Watch a device's log

```
onerom monitor log
```

Prints the One ROM's firmware and plugin logging as it is written, and starts with whatever it has logged since anything last listened — so the boot log is still there when you attach. Needs a running One ROM with the USB system plugin. Add `--output <FILE>` to keep a transcript, or use `onerom program --config c64.json --follow` to go straight from programming to watching. See [monitor log](#).

Erase a device

Erase flash. This is best done while stopped; by default the command reboots the device into the required state first. A fully erased unit falls back to One ROM's bootloader and is then reprogrammed with `--unrecognised` + `--board`, as [Recovering a bricked One ROM](#) describes:

```
onerom control erase --all
onerom control erase --offset 0x20000 --length 0x1000
```

Read [control erase](#) before using it while the device is running — erasing the core firmware or system plugin will take down the USB stack.

Prepare a 16-bit ROM image

16-bit ROM types (e.g. 27C400) may need their byte pairs swapped to match the order One ROM expects. Either rewrite the file first:

```
onerom image swap-bytes --input kick.bin --output kick-swapped.bin
```

or let the build do it, leaving the source file untouched:

```
onerom program --slot file=kick.bin,type=27C400,transform=swap_bytes
```

If your image interleaves several devices in one file — a 32-bit ROM set, say — `deinterleave` extracts a single lane. See [Image transforms](#) for the full set and how they compose.

Device states

Many commands reboot the device and, by default, pause briefly afterwards to let it re-enumerate on the USB bus.

- **Running** (default reboot target) — firmware active, serving ROMs.
- **Stopped** — One ROM/RP2350 bootloader (BOOTSEL), required for some flash operations.

Common controls, where a command supports them:

- `--running (-r)` / `--stopped (-p)` — choose the post-operation state.
- `--no-reboot` — leave the device as-is.
- `--fast` — skip the re-enumeration pause.
- `--msd (-m)` — mount the mass-storage device when rebooting into stopped mode.

Global behaviour worth knowing

- `--yes` (`-y`) auto-confirms all prompts (non-interactive use). It also suppresses the confirmation otherwise required for CPU frequencies above 150 MHz and voltages above 1.10 V in slot specs. Use with care.
- `--verbose` (`-v`) prints device-selection progress and other detail.
- `--log-level` `<LEVEL>` sets log verbosity; defaults to `warn` . Run `onerom --help` for the accepted levels.

CLI Reference

Synopsis

```
onerom [GLOBAL OPTIONS] <COMMAND> [ARGS]
```

Global options

Available on every command (they are `global` in clap terms and may appear at any level).

Option	Description
<code>--serial, -s</code> <code><DEVICE></code>	Select a One ROM by serial number. Required when multiple are connected; auto-selected when exactly one is present. Accepts <code>*</code> and <code>?</code> wildcards.
<code>--vid-pid <VID:PID></code> (alias <code>--id</code>)	USB vendor/product ID pair in hex (e.g. <code>1234:abcd</code>). Repeatable; when given, only these pairs are matched. Use with <code>--unrecognised</code> .
<code>--unrecognised, -u</code> (alias <code>--unrecognized</code>)	Allow management of unrecognised/unprogrammed/bricked RP2350 boards. The unit must still answer on its picoboot USB interface — a device that answers nothing is ignored either way. Use with caution — permits programming any attached RP2350 board. See Recovering a bricked One ROM .
<code>--yes, -y</code>	Auto-confirm all prompts. Also suppresses the over-limit CPU frequency/voltage confirmations.
<code>--verbose, -v</code>	Enable verbose output.
<code>--log-level <LEVEL></code>	Set log level. Defaults to <code>warn</code> .
<code>--version, -V</code>	Print version.
<code>--help, -h</code>	Print help. Works on any subcommand.

Most commands accept `--board ( -b )` to identify or override the board type, and rely on `--serial` (global) to pick a specific device.

Ice (STM32) boards are recognised, but the CLI cannot scan, program or build firmware for them. Where `--board` reaches a device or builds an image, naming an Ice board is an error rather than a later failure. Commands that only report hardware take them; each command's own entry below states what it accepts, and `board list` shows which boards are which.

Command summary

Command	Purpose	Device required
<code>scan</code>	Discover connected One ROMs	No
<code>program</code>	Build and flash firmware to a One ROM	Yes

Command	Purpose	Device required
<code>inspect</code>	Read-only device state and information	Yes
<code>monitor</code>	Watch a running One ROM as it works	Yes
<code>control</code>	Transient (non-persistent) device actions	Yes
<code>update</code>	Persistent device modifications	Yes
<code>image</code>	ROM image file manipulation	No
<code>firmware</code>	Build, inspect and manage firmware binaries	Varies
<code>plugin</code>	List available plugins	No
<code>chips</code>	List supported chip types and their flash usage	No
<code>board</code>	List board types, or draw a board's pin header / socket	No
<code>self</code>	Check for and download new releases of the CLI itself	No
<code>peek</code>	Alias for <code>inspect peek live</code>	Yes
<code>poke</code>	Alias for <code>control poke live</code>	Yes
<code>reboot</code>	Alias for <code>control reboot</code>	Yes

scan

Discover and list connected One ROMs — serial, USB location, name, board type, MCU and loaded firmware version. With `--verbose (-v)`, each device also shows its MCU variant and chip ID.

```
onerom scan
onerom scan --board fire-24-e
onerom scan --slots
```

Option	Description
<code>--board <BOARD></code>	Only show devices matching this board type. Conflicts with <code>--list-boards</code> . Must be a Fire board — a scan cannot find an Ice board.
<code>--list-boards</code>	List the known board types, the same listing as <code>board list</code> .
<code>--slots</code> (alias <code>--slot</code>)	Also show the ROM slot contents for each device found. Conflicts with <code>--list-boards</code> .

Example output:

```
Scanning ...
found 1 connected device:
  One ROM Fire 28 C - Firmware: v0.7.2 State: Running Serial: FC9D67248E8E8023
```

Device required: no.

program

Build a firmware image (from a config file, inline `--slot` specs, or a supplied binary) and flash it to a connected One ROM. This is the primary workflow. `onerom firmware program` is an alias for this command.

```
onerom program --config c64.json
onerom program --serial '5*' --config c64.json
onerom program --board fire-24-e \
  --slot file=kernal.bin,type=2364,cs1=active-low \
  --slot file=basic.bin,type=2364,cs1=active-low
onerom program --firmware firmware.bin
onerom program --config c64.json --out firmware.bin
```

Source of the firmware (mutually exclusive groups)

Option	Description
<code>--config, -j <FILE></code> (aliases <code>--config-file, --config-json, --json</code>)	ROM configuration JSON file. Conflicts with <code>--slot, --config-name, --config-description, --save-config, --no-config, --firmware</code> .
<code>--slot <SPEC></code> (alias <code>--rom</code>)	ROM slot specification; repeatable. See ROM slot specification . Conflicts with <code>--config, --no-config, --firmware</code> .
<code>--firmware <FILE></code> (alias <code>--fw</code>)	Flash a pre-built complete firmware binary directly. Conflicts with <code>--config, --slot, --base-firmware</code> and <code>--plugin</code> because a pre-built firmware already contains all ROMs/plugins. Also conflicts with <code>--version</code> .
<code>--base-firmware <FILE></code>	Use a local minimal firmware instead of downloading. With <code>--slot</code> , ROMs are built into it; alone, requires <code>--no-config</code> . Must be built with <code>EXCLUDE_METADATA=1</code> and <code>ROM_CONFIGS=</code> . Conflicts with <code>--firmware, --version</code> .
<code>--no-config</code>	Confirm flashing a base firmware with no ROM configuration. Only valid with <code>--config-name</code> and/or <code>--config-description</code> . Conflicts with <code>--config, --slot, --firmware</code> , and the <code>config-override</code> options below.

Configuration metadata

Option	Description
<code>--plugin <SPEC></code>	Plugin specification; repeatable. See Plugin specification . May be combined with <code>--config</code> : the plugins are inserted ahead of the config's ROM slots (which shift up), and it is an error if the config already defines a plugin of its own. Conflicts with <code>--firmware</code> .
<code>--config-name <NAME></code>	Name for the generated ROM configuration. Conflicts with <code>--config</code> .
<code>--config-description <DESC></code> (aliases <code>--desc, --description</code>)	Description for the generated configuration. Defaults to <i>"Created by the One ROM CLI"</i> . Conflicts with <code>--config</code> .

Option	Description
<code>--save-config <FILE></code>	Save the generated configuration to JSON. Only valid with <code>--slot</code> or <code>--no-config</code> . Conflicts with <code>--config</code> .

Per-device overrides

These are rejected with `--no-config`.

Option	Description
<code>--instance-name <NAME></code> (aliases <code>--name</code> , <code>--onerom</code> , <code>--one-rom</code> , <code>--onerom-name</code> , <code>--one-rom-name</code> , <code>--instance_name</code>)	Give this One ROM a name.
<code>--serial-override <SERIAL></code>	Override the device's reported serial number.
<code>--logging [B00L]</code> (aliases <code>--boot-logging</code>)	Enable boot logging. Takes an optional boolean; bare flag means <code>true</code> .
<code>--disable-swd [B00L]</code> (aliases <code>--swd-disable</code>)	Shut SWD down before ROM serving starts, so debug port accesses to SRAM don't steal cycles from the serving DMAs. SWD is available for the whole of boot — including boot logging — and goes off until the next reset. Nothing is logged past that point, and plugins get no logging. This is not a debug lockout: the boot ROM runs before the One ROM firmware does, and BOOTSEL/PICOBOT are unaffected. Optional boolean; bare flag means <code>true</code> .
<code>--turbo-boot [B00L]</code>	Enable turbo boot — starts serving faster by not reading the image select jumpers, so the first non-plugin slot is always the one served. More than one non-plugin slot is refused unless <code>--force</code> is given. Optional boolean; bare flag means <code>true</code> .

Board, version and output

Option	Description
<code>--board, -b <BOARD></code>	Target board type. Inferred from the connected device if omitted.
<code>--version <VERSION></code>	Firmware version to build against. Defaults to the latest release. Conflicts with <code>--firmware</code> , <code>--base-firmware</code> .
<code>--output, -o <FILE></code> (alias <code>--out</code>)	Also write the built firmware to this file while flashing.

Reboot and flashing behaviour

Option	Description
<code>--stopped, -p</code>	After flashing, reboot into stopped (bootloader) mode. Conflicts with <code>--running</code> .
<code>--running, -r</code>	After flashing, reboot into running mode (the default). Conflicts with <code>--stopped</code> .
<code>--no-reboot</code>	Do not reboot after flashing. Conflicts with <code>--stopped</code> .

Option	Description
<code>--fast</code>	Skip the re-enumeration pause after the final reboot. Conflicts with <code>--no-reboot</code> .
<code>--msd, -m</code>	Mount mass storage when rebooting into stopped mode.
<code>--verify</code>	Verify flash by reading back after programming.
<code>--force, -f</code>	Continue despite non-fatal problems: assembled firmware parse errors, a board type mismatch, and config warnings such as turbo boot with more than one non-plugin ROM slot. Each is reported as a warning instead.
<code>--batch</code> (aliases <code>--multiple, --multi</code>)	Program multiple devices, pausing for confirmation between each. Every board is programmed with the same configuration as the first.
<code>--scan-slots</code>	After programming, run <code>onerom scan --slots</code> to show the result. Conflicts with <code>--fast</code> .
<code>--follow</code>	After programming, monitor the One ROM's log, as <code>monitor log</code> does. Runs after <code>--scan-slots</code> , and only once the One ROM is back on the USB bus, so it shows the boot log of the firmware just flashed. Refused before anything is flashed if the image has no USB system plugin, since such a One ROM leaves the bus as soon as it serves. Conflicts with <code>--fast</code> , <code>--stopped</code> , <code>--no-reboot</code> and <code>--batch</code> .
<code>--reset-host</code> <PIN> (alias <code>--host-reset</code>)	After programming, pulse this pin low to reset the host system, as <code>control reset</code> does. Named as <code>gpio<N></code> or as a header pad (see <code>Pin values</code>). Runs after <code>--scan-slots</code> and before <code>--follow</code> , once the One ROM is back on the USB bus, and for each device in a <code>--batch</code> . The pulse is 100ms; use <code>control reset</code> for a different hold. Conflicts with <code>--fast</code> , <code>--stopped</code> and <code>--no-reboot</code> .

Device required: yes.

inspect

Read-only inspection of a connected One ROM.

```
onerom inspect <COMMAND>
```

Subcommand	Purpose	Device required
info	Identity and configuration	Yes
telemetry	Runtime telemetry (not yet supported)	Yes
slots	List ROM slots	Yes
image	Read a slot's ROM image (not yet supported)	Yes
peek	Read SRAM or the live ROM image	Yes
gpio	Show what each GPIO is and what it is doing	Yes (running)
header	Draw the device board's pin header	Yes
socket	Draw the device board's ROM socket pinout	Yes

inspect info

Show the device's serial number, user-assigned name, board type, MCU, firmware version and hardware revision. With `--verbose` (`-v`), also shows the MCU variant and chip ID.

```
onerom inspect info
onerom --serial 1234abcd inspect info
```

inspect telemetry

Access counts, timing statistics and other runtime metrics. **(not yet supported)**

Option	Description
<code>--json</code>	Output telemetry as JSON.

inspect slots

List the ROM image slots stored on the device — index, ROM type, size and description — marking the active slot. No options.

inspect image

Read (or save) the ROM image from a slot. **(not yet supported)**

Option	Description
<code>--slot <INDEX></code>	Slot index to read. Reads the active slot if omitted.
<code>--output, -o <FILE></code> (alias <code>--out</code>)	Save the image data to this file.

inspect peek

Read device memory. `peek memory` reads SRAM (and, in stopped state, page-aligned flash); `peek live` reads the ROM image currently being served.

```
onerom inspect peek <COMMAND>
```

inspect peek live

Read from the live ROM image at a **logical** ROM offset (starting at 0). The device must be running. Also reachable as the top-level `peek`.

```
onerom inspect peek live --address 0x100 --length 64
onerom inspect peek live --address 0 --length 8192 --output rom-image.bin
```

Option	Description
<code>--address, -a <ADDRESS></code> (alias <code>--addr</code>)	Logical ROM address to read from, starting at 0. Decimal or 0x hex. Default 0.
<code>--length, -l <LENGTH></code> (aliases <code>--len</code> , <code>--size</code>)	Number of bytes to read. Decimal or hex. If omitted, reads to the end of the live image.
<code>--output, -o <FILE></code> (alias <code>--out</code>)	Save the data to this file.

inspect peek memory

Read One ROM's SRAM. Most addresses reachable via PICOBOT can be queried. In stopped state, SRAM holds no meaningful data, and flash reads must be aligned to flash page boundaries.

```
onerom inspect peek memory --address 0x20000000 --length 128
onerom inspect peek memory --address 0x10000000 --length 8192 --output flash-start.bin
```

Option	Description
<code>--address, -a <ADDRESS></code> (alias <code>--addr</code>)	Address to read from. Decimal or 0x hex.
<code>--length, -l <LENGTH></code> (aliases <code>--len</code> , <code>--size</code>)	Number of bytes to read. Decimal or hex.
<code>--output, -o <FILE></code> (alias <code>--out</code>)	Save the data to this file.

inspect gpio

Show, one row per MCU GPIO, what that GPIO is on this board and what One ROM is currently doing with it.

The device must be **running** with the USB system plugin: One ROM's own command handler lives in that plugin, and a stopped device is in the RP2350 bootloader where it does not exist. See [Device states](#).

```

onerom inspect gpio
onerom inspect gpio --all
onerom inspect gpio --pin gpio9
onerom inspect gpio --pin x1

```

Option	Description
<code>--pin <PIN></code>	Show only this pin, named as <code>gpio<N></code> or as a header pad (see Pin values). Conflicts with <code>--all</code> .
<code>--board <BOARD></code>	Board type, overriding what the device reports. Only needed to resolve a <code>--pin</code> pad name on a board this build does not recognise.
<code>--all</code>	Also list GPIOs with no function at all. By default only GPIOs connected to something are shown.

By default the table lists only the GPIOs connected to **something** — a ROM socket signal, a board peripheral or a header pad. On a 48-GPIO board a quarter of the GPIOs are connected to nothing, and listing them buries the rows worth reading; a line beneath the table says how many were omitted. `--all` lists every GPIO. Note the filter is on what the GPIO *is*, not on what the device reports using it for: the `X1 / X2` and image-select pads report `free` and are exactly what you read this table to find, so they always appear.

The number of GPIOs the device has is its own — 30 on an RP2350A, 48 on an RP2350B.

Columns:

Column	Meaning
GPIO	MCU GPIO number.
Function	Everything this GPIO is, comma-separated in a fixed order: its ROM socket signal under the image being served (A5, D3, CS1, BYTE/VPP), then the board peripheral (Status LED, RGB LED, USB VBUS, ext flash CS), then the header pad (X1, X2, SEL_A). – if the GPIO is connected to nothing.
Dir	out if the pin's output driver is enabled, in if not.
Level	The GPIO's level, 0 or 1: what an out pin is driving, what an in pin reads.
Max V	5V if the GPIO is 5V-tolerant, 3V3 if it is an RP2350 ADC pin and therefore 3.3V-only, ? if the board is not characterised.
One ROM use	What One ROM itself is using the GPIO for: free, serving (read), serving (driven) or system.

`Function` lists everything that applies rather than stopping at the first match, so a GPIO that is genuinely two things says so: on a `fire-24-f` the Status LED and the RGB LED are the same GPIO, and it reads `Status LED, RGB LED`. Names that would repeat are shown once — on a 32-pin board a high address line is both the socket's `A17` and the `A17` header pad, which is one net.

`Function` names only what a **GPIO** is. A header pad may carry more than the GPIO behind it — on a Fire 24/28 board the `SEL_C` and `SEL_D` pads sit on the SWCLK and SWDIO nets — but SWCLK and SWDIO are dedicated RP2350 pins with no GPIO of their own, so they do not appear here. Run `inspect header` for the pad-by-pad view, which shows every role each pad carries.

Only `Dir`, `Level` and `One ROM use` come from the device. `Function` is derived by the CLI from the board's pin map and the chip type being served: the device deliberately reports what taking a pin over would cost, never what the pin *is*. `serving (read)` pins (address, chip-select, `/BYTE`) can be driven and released; `serving (driven)` pins (the data pins) cannot be given back without a reboot — see `control pin`.

With `--verbose (-v)` the table is followed by a legend restating where each column comes from, what `Dir` and `Level` mean and what the `3V3 / 5V` tags mean. Nothing is lost without it: the cost of taking a serving pin over is stated at the point of action by `control pin` itself.

A board revision or ROM type this build does not recognise costs the derived names, not the listing: `Function` falls back to `-` (or, for a socket pin whose chip type is unknown, `socket pin <N>`), and with no recognised board at all nothing is filtered out, since nothing can be ruled out. On a board with no pin-header descriptor, pad names come from the board's pin assignments alone and `--verbose` says so beneath the table.

On a Fire 28 (rev C) serving a 27512:

One ROM Fire 28 C – Firmware: v0.7.2 State: Running Serial: 2E4A671D1C92AE5C

GPIO state · One ROM Fire 28 (rev C) · RP235xB · serving 27512

GPIO	Function	Dir	Level	Max V	Current use
0	D0	out	0	5V	serving (driven)
1	D1	out	0	5V	serving (driven)
2	D2	out	1	5V	serving (driven)
3	D3	out	1	5V	serving (driven)
4	D4	out	1	5V	serving (driven)
5	D5	out	1	5V	serving (driven)
6	D6	out	1	5V	serving (driven)
7	D7	out	0	5V	serving (driven)
8	X2	in	0	5V	free
9	X1	in	0	5V	free
10	CE/PE	in	0	5V	serving (read)
11	OE/VPP	in	0	5V	serving (read)
12	A14	in	0	5V	serving (read)
13	A10	in	0	5V	serving (read)
14	A11	in	0	5V	serving (read)
15	A9	in	0	5V	serving (read)
16	A8	in	0	5V	serving (read)
17	A13	in	0	5V	serving (read)
18	A15	in	0	5V	serving (read)
19	A12	in	0	5V	serving (read)
20	A7	in	0	5V	serving (read)
21	A6	in	0	5V	serving (read)
22	A5	in	0	5V	serving (read)
23	A4	in	0	5V	serving (read)
24	A3	in	0	5V	serving (read)
25	A2	in	0	5V	serving (read)
26	A1	in	0	5V	serving (read)
27	A0	in	0	5V	serving (read)
38	SEL_C	in	1	5V	free
39	SEL_D	in	1	5V	free
40	SEL_A	in	0	3V3	free
41	SEL_B	in	0	3V3	free
44	RGB LED	out	0	3V3	system
45	Status LED	out	0	3V3	system
46	USB VBUS	in	1	3V3	system

13 GPIOs with no function are hidden – use --all to show them.

inspect led

Show what the status LED is doing now — the mode it is in, how fast it is running, and which GPIO it is on. Use `inspect rgb` for the RGB LED some models carry.

```
onerom inspect led
```

On a `fire-28-c` running `onerom control led flame --period 900`:

```
Status LED:
  Mode:      flame
  Period:    900ms
```

`Period` appears only for the modes that repeat.

`--verbose` adds the GPIO the LED is on, and says so where the board wires both LEDs to one pin — a `fire-24-f` does, a `fire-28-c` does not.

No options. Device required: yes, and it must be running with the USB system plugin.

Needs One ROM firmware v0.7.2 or later with the v0.2.2 or later USB system plugin. An older One ROM says so rather than reporting something invented.

inspect rgb

Show what the RGB LED is doing now — the mode, the colour, the brightness, how fast it is running, and which GPIO it is on.

```
onerom inspect rgb
```

On a `fire-28-c` running `onerom control rgb breathe --colour cyan --brightness 60 --period 4000`:

```
RGB LED:
  Mode:      breathe
  Colour:    #00FFFF (cyan)
  Brightness: 60%
  Period:    4000ms
```

A colour is named where it is one of the names `--colour` accepts. One that is not prints as hex alone — `#7F3C22`.

Each repeating mode has a shortest period it can run at, and a shorter one is refused rather than quietly run at the minimum:

Mode	Shortest period
cycle, breathe	1000ms
flame	500ms
beacon, blink	50ms

`cycle` walks the hues itself rather than showing a colour you set, so no `Colour` is reported while it runs:

```
RGB LED:
  Mode:      cycle
  Brightness: 25%
  Period:    3000ms
```

`--verbose` adds the GPIO:

```

RGB LED:
  Mode:      cycle
  Brightness: 25%
  Period:    3000ms
  GPIO:      44

```

Only some One ROM models have an RGB LED. On a board without one this reports:

```
RGB LED: this board does not have one
```

Where the RGB LED and the status LED share a GPIO — as they do on a `fire-24-f` — both commands report the same pin and say that it is shared. Both LEDs still work, and no mode is restricted.

No options. Device required: yes, and it must be running with the USB system plugin.

Needs One ROM firmware v0.7.2 or later with the v0.2.2 or later USB system plugin.

inspect header

Draw the connected device's pin (jumper / programming) header as ASCII. The board is inferred from the device. This is the device-oriented form of `board header`; see there for what the diagram shows.

```
onerom inspect header [--board <board>]
```

Option	Description
<code>--board,</code> <code>-b</code>	Board type, overriding what the connected One ROM reports. Only needed on a One ROM whose board type this build does not recognise.

`--board` is an override, not a substitute for the device: this command draws the board of a *connected* One ROM, so one must still be present. To draw a board by name with nothing connected, use `board header`.

inspect socket

Draw the connected device's ROM socket pinout as ASCII. The board is inferred from the device. This is the device-oriented form of `board socket`.

```
onerom inspect socket [--board <board>] [--chip-type <chip>] [--gpio]
```

Option	Description
<code>--board,</code> <code>-b</code>	Board type, overriding what the connected One ROM reports.
<code>--chip-type <chip>,</code> <code>-c</code>	Label pins with this ROM type's functions instead of GPIOs, and report the chip's image size on this board.
<code>--gpio</code>	Overlay the GPIO(s) onto the <code>--chip-type</code> function view (requires <code>--chip-type</code>).

As with `inspect header`, `--board` overrides the connected One ROM's reported board type rather than standing in for the device.

monitor

Watch a running One ROM as it works.

```
onerom monitor <COMMAND>
```

Subcommand	Purpose	Device required
log	Show the One ROM's log as it is written	Yes

monitor log

Attach to the One ROM's USB serial port and print the firmware and plugin logging it sends, until the One ROM is disconnected, rebooted or stopped, or you press Ctrl-C.

Every attach opens with the One ROM naming itself, in a block headed `----- One ROM USB log -----`. What it has logged since anything last listened arrives after that, so attaching after a reboot still shows the boot log, which opens with a `-----` divider of its own:

```
Monitoring log - press Ctrl-C to stop
----- One ROM USB log -----
One ROM fire-28-c v0.7.2
Serial: 2E4A671D1C92AE5C
Logging: boot, plugin-internal, error, plugin-application
-----
-----
One ROM v0.7.2.1 https://onerom.org
Copyright (c) 2026 Piers Finlayson <piers@piers.rocks>
Built: Aug 15 2026 14:09:53
Commit: 5db495a
-----
RP235XB
RAM: 520KB
Flash: 2048KB
Freq: 150MHz
```

`Name:` appears only when the One ROM has an instance name set, and `Logging:` lists the kinds of output switched on — see [Logging](#). A One ROM that cannot forward its log says so there, rather than leaving you watching a silent port.

```
onerom monitor log
onerom --serial 1234abcd monitor log
onerom monitor log --output boot.txt
```

Option	Description
--output, -o <FILE> (alias --out)	Also write the One ROM's output to this file, replacing its contents. The file receives what the One ROM sends and nothing else, so it is a transcript of the device rather than of this command. The output still appears on screen as well.

The One ROM's output goes to stdout and everything this command says about itself goes to stderr, so `onerom monitor log > boot.txt` captures the log on its own. `--verbose` adds a line naming the serial port, for when you want to point another tool at it.

What this command needs, and what it cannot do:

- The One ROM must be **running**, and must have been programmed with the USB system plugin. That plugin provides the serial port and forwards the log into it.
- Nothing is forwarded until this command — or another terminal — opens the port. A One ROM nothing is listening to accumulates its log rather than discarding it, which is why the boot log is still there when you attach.
- A debug probe reading the log over SWD consumes the same bytes. With both running the stream is split arbitrarily between them and neither sees all of it, so use one at a time.
- If nothing arrives within two seconds the command fails. Since every attach begins with the banner above, a One ROM with a current USB plugin always sends something — so a timeout points at a plugin too old to forward the log at all.

Device required: yes.

control

Transient actions on a connected One ROM. These affect current state but do not persist across power cycles.

```
onerom control <COMMAND>
```

Subcommand	Purpose	Device required
reboot	Reboot the device	Yes
led	Control the status LED	Yes
poke	Write to SRAM or the live ROM image	Yes
reset	Pulse a GPIO low to reset the host system	Yes (running)
select	Select the active ROM slot (not yet supported)	Yes
pin	Drive a pin high, low or high-impedance	Yes (running)
erase	Erase flash memory	Yes

control reboot

Restart the firmware; the device re-initialises and resumes serving. By default pauses afterwards for re-enumeration. Also reachable as the top-level `reboot`.

```
onerom control reboot
```

Option	Description
--stopped, -p	Reboot into stopped (bootloader) state.
--running, -r	Reboot into running (serving) state. Default.
--fast	Don't pause for re-enumeration.
--msd, -m	Mount mass storage when rebooting into stopped mode. Conflicts with --running.

`--stopped` and `--running` are mutually exclusive.

control led

Control the status LED — the single-colour LED every One ROM has. The RGB LED that some models carry is driven by `control rgb` instead.

```

onerom control led on
onerom control led off
onerom control led beacon --hold 10000
onerom control led flame --period 1200
onerom control led blink

```

Subcommand	Description
on	Turn the status LED on.
off	Turn the status LED off.
beacon	Beacon the LED to identify a physical unit.
flame	Flame effect on the LED.
blink	Blink the LED on and off until something changes it.

Option	Description	Subcommands
--hold <MS>	Stay in this mode for this many milliseconds, then go back to what the LED was doing before. The device times it, so it completes even if the command does not. Maximum 60000.	all
--period <MS>	Milliseconds for one repetition — one blink for beacon and blink, one pass of the flicker for flame. Defaults to 100, 1000 and 575 respectively. Minimum 50 for beacon and blink, 500 for flame.	beacon, blink, flame

`beacon` ends by itself after 2500ms unless `--hold` says otherwise. `blink` is the same on-and-off toggle but slower and unbounded — it runs until something changes it, or until a `--hold` you give it expires.

The status LED is lit or dark, so it takes no colour and no brightness. `cycle` and `breathe` are built out of a colour and are the two modes it cannot do.

`--hold` and `--period` need One ROM firmware v0.7.2 or later with the v0.2.2 or later USB system plugin. The CLI checks before sending, and says so rather than reporting success on a device that would ignore them. A plain `on`, `off`, `beacon` or `flame` works on any One ROM and costs no extra exchange with the device.

Device required: yes.

control rgb

Control the RGB LED that some One ROM models carry. For the single-colour status LED every model has, see [control led](#).

```

onerom control rgb on --colour red
onerom control rgb on --colour #FF8000 --brightness 40
onerom control rgb cycle --period 3000
onerom control rgb off

```

Subcommand	Description
on	Light the LED at a colour.
off	Turn the LED off.
beacon	Beacon the LED to identify a physical unit. Ends by itself after 2500ms.
flame	Flame effect on the LED.
cycle	Rotate through the hues.
breathe	Fade the colour up and down.
blink	Alternate the colour with dark.

Option	Description	Subcommands
<code>--colour <COLOUR></code> (alias <code>--color</code>)	A name, or hex written <code>#RRGGBB</code> or <code>0xRRGGBB</code> . Defaults to red.	all but off and cycle
<code>--brightness <PERCENT></code>	1 to 100. Omit for the device's default, which is deliberately modest — an RGB LED at full output is uncomfortable at desk distance.	all but off
<code>--period <MS></code>	Milliseconds for one repetition.	beacon, flame, cycle, breathe, blink
<code>--hold <MS></code>	Stay in this mode for this many milliseconds, then go back to what the LED was doing before. The device times it, so it completes even if the command does not. Maximum 60000.	all

The named colours are red, green, blue, white, yellow, cyan, magenta, orange, purple and pink. `cycle` chooses its own colours, so it takes no `--colour`.

Each repeating mode has a shortest period it can run at, and a shorter one is refused rather than quietly run at the minimum:

Mode	Default period	Shortest period
cycle, breathe	5000ms	1000ms
flame	575ms	500ms
blink	1000ms	50ms
beacon	100ms	50ms

Nothing is printed unless the CLI is verbose:

```
$ onerom --verbose control rgb on --colour orange --brightness 40
RGB LED on
```

Read back what the LED is doing with `inspect rgb`:

```

RGB LED:
  Mode:      on
  Colour:    #FF6000 (orange)
  Brightness: 40%

```

Only some One ROM models have an RGB LED. On a board without one, these commands say so rather than appearing to work. Needs One ROM firmware v0.7.2 or later with the v0.2.2 or later USB system plugin.

Device required: yes, and it must be running with the USB system plugin.

control poke

Transient writes to device memory — changes are lost on reboot. Use `update` for persistent flash writes.

```
onerom control poke <COMMAND>
```

control poke memory

Write a single byte or a binary file to SRAM at a given address. When the device is running, virtual addresses are available (e.g. `0x90000000` is the start of the live ROM image — though prefer `poke live` for that). Writing arbitrary SRAM can corrupt firmware state.

```

onerom control poke memory --address 0x20000010 --byte 0xFF
onerom control poke memory --address 0x20000000 --input patch.bin

```

Option	Description
<code>--address, -a <ADDRESS></code> (alias <code>--addr</code>)	Address to write to. Decimal or 0x hex.
<code>--byte <BYTE></code> (alias <code>--value</code>)	Single byte value to write. Decimal or hex.
<code>--input, -i <FILE></code> (alias <code>--in</code>)	Write the contents of this binary file.

Exactly one of `--byte` / `--input` is required.

control poke live

Write a single byte or a binary file to the live ROM image at a **logical** ROM offset (starting at 0). Useful for patching a running ROM without reflashing. Also reachable as the top-level `poke`.

```

onerom control poke live --address 0x100 --byte 0xEA
onerom control poke live --address 0 --input patch.bin

```

Option	Description
<code>--address, -a <ADDRESS></code> (alias <code>--addr</code>)	Logical ROM address, starting at 0. Decimal or 0x hex. Default 0.
<code>--byte <BYTE></code> (alias <code>--value</code>)	Single byte value to write. Decimal or hex.
<code>--input, -i <FILE></code> (alias <code>--in</code>)	Write the contents of this binary file.
<code>--delta</code> (alias <code>--deltas</code>)	Only write bytes that differ from current device content. Requires <code>--input</code> .

Option	Description
<code>--dry-run</code> (alias <code>--dryrun</code>)	Show what would be written without writing. Requires <code>--delta</code> .

Exactly one of `--byte` / `--input` is required.

control reset

Pulse a GPIO low, then release it, to reset the host system One ROM is installed in — useful in scripted workflows after programming a new image. `program --reset-host` does the same thing as the last step of programming, and is the shorter way to say it.

`--pin` is the pin your reset wire is soldered to, typically an image-select pad whose jumper has been removed — `sel_c` is the usual choice, as more boards have it than have X pads and it is 5V tolerant where it exists — or an X1 / X2 pad. Name it by pad (`sel_c` , `x1`) or by MCU GPIO (`gpio9`) — see [Pin values](#). `inspect header` shows which GPIO is behind each pad.

The line is only ever **driven low and then released to high impedance**. A reset net has its own pull-up and may have other drivers on it, so there is deliberately no way to drive it high. Use `control pin` if you need arbitrary states.

The **device** times the pulse, not the CLI: if this command is interrupted, the terminal closes or the cable is pulled mid-pulse, the device still releases the pin. The device's own limit is 60 seconds.

The device must be **running** with the USB system plugin — see [Device states](#).

```
onerom control reset --pin sel_c
onerom control reset --pin gpio9
onerom control reset --pin gpio9 --hold 500
```

Option	Description
<code>--pin <PIN></code>	Pin the reset wire is connected to, named as <code>gpio<N></code> or as a header pad (see Pin values). Required.
<code>--board <BOARD></code>	Board type, overriding what the device reports. Only needed to resolve a <code>--pin</code> pad name on a board this build does not recognise.
<code>--hold <MS></code>	Milliseconds to hold reset asserted. Decimal or 0x hex. Default 100; 0 is rejected, because a reset pulse with no end is not a reset.

If One ROM is itself using the GPIO the command is refused, naming what it is doing; `control reset` has no `--force` of its own, and the message points at `control pin --force` for the case where that is genuinely what you want. If the GPIO is not 5V-tolerant the command warns and asks for confirmation, which `--yes` answers.

The pulse counts toward the device's limit on pins under a timed hold at once — see `control pin`.

```
$ onerom control reset --pin x1
Asserted reset on x1 (gpio9) for 100ms - the device times the pulse and releases the pin
```

control select

Switch the device to serving the specified slot immediately (not persistent). **(not yet supported)**

Option	Description
<code>--slot <INDEX></code>	Slot index to activate. Required.

control pin

Drive a One ROM pin high, low or high-impedance, optionally for a bounded period.

`--pin` names an MCU GPIO or a header pad (see [Pin values](#)); the command is named for what is being addressed rather than for any one spelling.

Without `--hold` the state is latched until something else changes it. With `--hold` the **device** holds the state for that many milliseconds and then applies `--then` — high impedance unless you say otherwise. As with `control reset`, the hold is timed on the device, so an interrupted CLI cannot leave a pin latched.

A limited number of pins can have a timed `--hold` at once. A pin driven without `--hold` is latched indefinitely and is not included in that limit. One ROM rejects a hold on a further pin once the limit is reached.

The device must be **running** with the USB system plugin — see [Device states](#). `inspect gpio` shows what each GPIO is and what One ROM is using it for.

```
onerom control pin --pin gpio9 --state high
onerom control pin --pin gpio9 --state low --hold 250
onerom control pin --pin gpio9 --state low --hold 250 --then high
onerom control pin --pin x1 --state 0 --hold 250
onerom control pin --pin sel_a --state z
```

Option	Description
<code>--pin <PIN></code>	Pin to drive, named as <code>gpio<N></code> or as a header pad (see Pin values). Required.
<code>--board <BOARD></code>	Board type, overriding what the device reports. Only needed to resolve a <code>--pin</code> pad name on a board this build does not recognise.
<code>--state <STATE></code>	high, low, or z (high-impedance). 1 and 0 are accepted for high and low. Required.
<code>--hold <MS></code>	Hold <code>--state</code> for this many milliseconds, then apply <code>--then</code> . Decimal or 0x hex. Omit to latch indefinitely. The device's own limit is 60 seconds.
<code>--then <STATE></code>	State to apply when <code>--hold</code> expires: high, low or z (or 1/0). Default z. Requires <code>--hold</code> .
<code>--force</code>	Drive the GPIO even though One ROM is using it for serving.

Refusals and warnings. If One ROM is itself using the GPIO, the command is refused and names what it is doing. `--force` overrides, and prints what that costs:

- a pin serving **reads** (address, chip-select, /BYTE) is reversible — serving keeps reading it, and `--state z` puts it back;
- a pin serving **drives** (a data pin) is not — forcing it takes the pin away from the PIO that drives it, and serving stays broken until the device is rebooted.

If the GPIO is not 5V-tolerant — an RP2350 ADC pin, per the board metadata, not a measurement — the command warns and asks for confirmation, which `--yes` or `--force` answers. Nothing else about the pad is checked: what is wired to it, whether a jumper is fitted and what voltage the far end sits at are yours to know.

```
$ onerom control pin --pin x1 --state low --hold 2000
Set x1 (gpio9) low for 2000ms - the device times the hold and then sets it high impedance
```

control erase

Permanently erase flash contents — firmware, metadata and ROM images. A fully erased unit boots into One ROM's bootloader and is reprogrammed with `--unrecognised` + `--board`.

Best performed while stopped; by default the command reboots into the required state first. Erasing the core firmware or the system plugin while **running** takes down the USB stack (requiring **manual BOOTSEL**), and large erases may cause a temporary USB drop and re-enumerate — in which case the erase likely succeeded and can be checked with `inspect peek memory`. Anything else running from flash (e.g. a user plugin) may crash during an erase.

Offsets are relative to the flash base `0x10000000`. Ranges must be 4096-aligned. Multiple ranges may be erased in one operation.

```
onerom control erase --all
onerom control erase --offset 0x20000 --length 0x1000
```

Option	Description
<code>--all, -a</code>	Erase all flash contents.
<code>--offset <OFFSET></code>	Erase at offset(s) from the flash base. 4096-aligned; pair each with a <code>--length</code> ; repeatable. Conflicts with <code>--address</code> .
<code>--address <ADDRESS></code> (alias <code>--addr</code>)	Erase at absolute address(es). 4096-aligned; pair each with a <code>--length</code> ; repeatable. Conflicts with <code>--offset</code> .
<code>--length <LENGTH></code> (aliases <code>--len, --size</code>)	Length of each range. 4096-aligned; specify once per <code>--offset/--address</code> ; repeatable. Conflicts with <code>--all</code> .
<code>--no-reboot, -n</code>	Don't reboot before or after erasing. Risky if One ROM is accessing the range.
<code>--stopped, -p</code>	Reboot into stopped mode after erasing.
<code>--running, -r</code>	Reboot into running mode after erasing.

Option	Description
<code>--msd, -m</code>	Mount mass storage when rebooting into stopped mode. Requires <code>--stopped</code> .
<code>--fast</code>	Don't pause for re-enumeration. Requires a reboot mode.

One of `--all` / `--offset` / `--address` is required. `--stopped` and `--running` are mutually exclusive, and both conflict with `--no-reboot`.

update

Persistent modifications — these write to flash and survive power cycles.

```
onerom update <COMMAND>
```

Subcommand	Purpose	Device required
slot	Write a ROM image to a flash slot (not yet supported)	Yes
commit	Commit the live image to flash (not yet supported)	Yes
otp	Read/write OTP memory (not yet supported, hidden)	Yes

update slot

Write a ROM image to a flash slot; persists across power cycles. The ROM type and chip-select configuration must match the slot's existing configuration, or the slot must be empty. **(not yet supported)**

```
onerom update slot --slot 2 --image kernal.bin
```

Option	Description
--slot <INDEX>	Flash slot index to write. Required.
--image <FILE>	ROM image file to write. Required.

update commit

Persist the currently active RAM image to its corresponding flash slot. **(not yet supported)**

```
onerom update commit
onerom update commit --slot 2
```

Option	Description
--slot <INDEX>	Slot to commit. Commits the active slot if omitted.

update otp

Read or write RP2350 OTP memory, including One ROM-specific USB configuration and identity data. Hidden, advanced. **OTP writes are irreversible. (not yet supported)**

Option	Description
--read	Read and display OTP contents. Conflicts with --write.
--write <ROW=VALUE>	Write a value to an OTP row. Conflicts with --read.

image

ROM image file manipulation. No device connection required.

```
onerom image <COMMAND>
```

image swap-bytes

Swap adjacent byte pairs — reverses byte order within each 16-bit word throughout the image. Required for 16-bit ROM types (e.g. 27C400) when the source has the opposite byte order to what One ROM expects. The input must have an even number of bytes.

```
onerom image swap-bytes --input kick.bin --output kick-swapped.bin
```

Option	Description
<code>--input, -i <FILE></code> (alias <code>--in</code>)	Input ROM image file.
<code>--output, -o <FILE></code> (alias <code>--out</code>)	Output file path.

The same operation is available during a build as `--slot transform=swap_bytes` ; see [Image transforms](#).

Before writing, the input is checked against a list of known 16-bit ROM headers. Where it is recognised and swapping it would be incorrect to program with One ROM, a warning is printed and the swap still goes ahead:

```
$ onerom image swap-bytes --input kick-swapped.bin --output out.bin
Warning: kick-swapped.bin starts with an Amiga ROM header, low byte of each pair first.
  It is already the way One ROM needs it, and swapping the bytes will stop it
  working with One ROM.
Written to out.bin
```

See [16-bit ROM image byte ordering](#).

Device required: no.

image deinterleave

Extract one lane from an interleaved ROM image. The image contains `--stride` interleaved lanes of `--bytes` bytes each; lane `--offset` is kept and the rest discarded. Used to split a wide ROM image, distributed as a single interleaved file, into the narrower images each device needs.

The input length must be a multiple of `--bytes × --stride` ; the output is `1/--stride` of the input length.

```
# odd bytes of a 16-bit interleaved image
onerom image deinterleave --input rom16.bin --output odd.bin --offset 1 --stride 2

# byte 2 of a 32-bit interleaved image
onerom image deinterleave --input rom32.bin --output b2.bin --offset 2 --stride 4

# the upper 16-bit half of each 32-bit word
onerom image deinterleave --input rom32.bin --output hi.bin --offset 1 --stride 2 --bytes 2
```

Option	Description
<code>--input, -i <FILE></code> (alias <code>--in</code>)	Input ROM image file.
<code>--output, -o <FILE></code> (alias <code>--out</code>)	Output file path.
<code>--offset <N></code>	Which lane to keep. Must be less than <code>--stride</code> .
<code>--stride <N></code>	How many lanes the image interleaves. Must be at least 2.
<code>--bytes <N></code> (alias <code>--unit</code>)	Width of one lane, in bytes. Defaults to 1; use 2 to keep 16-bit words together.

The same operation is available during a build as `--slot transform=deinterleave:<offset>/<stride>[/<bytes>]` ; see [Image transforms](#).

Device required: no.

image convert

Convert a ROM image between formats. Reads `--input` in the `--from` format and writes `--output` in the `--to` format. Formats: `binary` (aliases `bin`, `raw`), `ihex` (Intel HEX; aliases `intel-hex`, `intel_hex`) and `srec` (Motorola S-record; aliases `s-record`, `s_record`, `srecord`, `motorola`, `s19`). Any format can be converted to any other, including `ihex` to `srec`. The format set is designed to grow — further formats can be added without changing the command.

```
onerom image convert --from ihex --to binary --input rom.hex --output rom.bin
onerom image convert --from binary --to srec --input rom.bin --output rom.s19 --load-address $E000
```

Option	Description
<code>--from <FORMAT></code>	Input format: <code>binary</code> , <code>ihex</code> or <code>srec</code> .
<code>--to <FORMAT></code>	Output format: <code>binary</code> , <code>ihex</code> or <code>srec</code> .
<code>--input, -i <FILE></code> (alias <code>--in</code>)	Input ROM image file.
<code>--output, -o <FILE></code> (alias <code>--out</code>)	Output file path.
<code>--load-address <ADDR></code>	Load address (decimal, or <code>0x/\$</code> -prefixed hex). Only valid when one side is <code>ihex</code> or <code>srec</code> ; subtracted when reading that format, used as the base when writing it. Defaults to 0.

Both record formats are written with 16-byte data records and a terminating record; unwritten addresses read as `0xFF` when decoding. S-record output uses one data record type throughout, the narrowest that addresses the whole image — `S1` below 64 KB, then `S2`, then `S3` — with the paired `S9 / S8 / S7` terminator. An S-record file that ends without a termination record is read, which is what `srec_cat` writes unless it is given an execution start address. Device required: no.

firmware

Build, inspect and manage firmware binaries. Use `program` to flash; `firmware build` produces a binary without flashing.

```
onerom firmware <COMMAND>
```

Subcommand	Purpose	Device required
<code>build</code>	Build a firmware binary from a config	No
<code>inspect</code>	Inspect a firmware binary	No
<code>releases</code>	List firmware releases	No
<code>download</code>	Download a release binary	No
<code>chips</code>	List supported chip types and their flash usage	No
<code>program</code>	Alias for <code>onerom program</code>	Yes

firmware build

Produce a flashable firmware binary for a board and MCU from a JSON config or inline `--slot` args, without flashing.

```
onerom firmware build --config c64.json --board fire-24-e --out firmware.bin
onerom firmware build --board fire-24-e \
  --slot file=kernal.bin,type=2364,cs1=active-low \
  --out firmware.bin
```

The configuration options mirror `program`: `--config (-j)`, `--slot`, `--plugin`, `--config-name`, `--config-description`, `--save-config`, `--no-config`, and the per-device overrides `--instance-name`, `--serial-override`, `--logging`, `--disable-swd`, `--turbo-boot` (all rejected with `--no-config`). Build-specific options:

Option	Description
<code>--board, -b <BOARD></code>	Target board type. Required when not inferrable from a connected device.
<code>--version <VERSION></code>	Firmware version to build against. Defaults to latest.
<code>--base-firmware <FILE></code>	Use a local minimal firmware instead of downloading. Must be built with <code>EXCLUDE_METADATA=1</code> and <code>ROM_CONFIGS=</code> . Conflicts with <code>--version</code> .
<code>--output, -o <FILE> (alias --out)</code>	Output file path. Defaults to <code>onerom-<board>-<version>.bin</code> . Conflicts with <code>--path</code> .
<code>--path <DIR></code>	Output directory, using the default filename. Conflicts with <code>--output</code> .

Option	Description
<code>--force, -f</code>	Continue despite non-fatal problems: assembled firmware parse errors, a board type mismatch, and config warnings such as turbo boot with more than one non-plugin ROM slot. Each is reported as a warning instead.

Device required: no.

firmware inspect

Show a firmware binary's version, board type, MCU, and embedded ROM images and metadata.

```
onerom firmware inspect --firmware firmware.bin
```

Option	Description
<code>--firmware <FILE></code> (aliases <code>--fw</code> , <code>--in</code> , <code>--input</code>)	Firmware binary to inspect.
<code>--board, -b <BOARD></code>	Inspect the release firmware for this board type. Conflicts with <code>--firmware</code> .
<code>--version <VERSION></code>	Firmware version to inspect. Defaults to latest. Conflicts with <code>--firmware</code> .

firmware releases

List available firmware releases with supported boards and MCUs.

```
onerom firmware releases
```

Option	Description
<code>--board, -b <BOARD></code>	Show only releases for this board type.
<code>--all, -a</code>	Show all releases even if a device is attached and detected. Conflicts with <code>--board</code> .

firmware download

Download the base (ROM-less) firmware binary for a version/board/MCU.

```
onerom firmware download --version 0.6.5 --board fire-24-e --out firmware.bin
```

Option	Description
<code>--version <VERSION></code>	Version to download. Defaults to latest.
<code>--board, -b <BOARD></code>	Target board type. Inferred from device if omitted.
<code>--output, -o <FILE></code> (alias <code>--out</code>)	Output file path. Defaults to <code>onerom_<board>_<version>.bin</code> . Conflicts with <code>--path</code> .
<code>--path <DIR></code>	Output directory, using the default filename. Conflicts with <code>--output</code> .

This firmware binary can then be used as a base for `firmware build` or flashed with `program` using the `--base-firmware` option. Do not flash it directly, as it contains no ROM configuration and will not serve any ROMs.

firmware chips

List the chip types a board can emulate and the flash each one uses, or all chip types grouped by pin count. Identical to the top-level `chips`.

```
onerom firmware chips --board fire-24-e
onerom firmware chips --board fire-24-e --chip-type 2364
onerom firmware chips --all
```

Option	Description
<code>--board, -b <BOARD></code>	Show supported chips for this board. Conflicts with <code>--all</code> .
<code>--all, -a</code>	Show all chips grouped by pin count. Conflicts with <code>--board</code> .
<code>--chip-type, -c <CHIP></code>	Show just this chip type's flash usage on the board. Conflicts with <code>--all</code> .

plugin

List available plugins from the release manifest, with versions and minimum firmware requirements. Without a connected device or `--fw-version` , minimum firmware requirements are shown for reference; with either, incompatible plugins are flagged.

```
onerom plugin
onerom plugin --all-versions
onerom plugin --type system
onerom plugin --fw-version 0.6.6
```

Option	Description
<code>--all-versions, -a</code>	Show all versions of each plugin, not just the latest.
<code>--type, -t <TYPE></code>	Filter by plugin type: system or user.
<code>--fw-version <VERSION></code>	Firmware version to check compatibility against.

Device required: no.

chips

List supported chip types — for a board, with the flash each one uses, or all grouped by pin count. Top-level alias for `firmware chips`.

```
onerom chips --board fire-24-e
onerom chips --board fire-24-e --chip-type 2364
onerom chips --all
```

Option	Description
<code>--board, -b <BOARD></code>	Show supported chips for this board. Conflicts with <code>--all</code> .
<code>--all, -a</code>	Show all chips grouped by pin count. Conflicts with <code>--board</code> .
<code>--chip-type, -c <CHIP></code>	Show just this chip type's flash usage on the board. Conflicts with <code>--all</code> .

Flash usage

For a board, each chip is listed with its **ROM size** (the chip's own capacity) and its **image size** — the flash One ROM uses to emulate it, which is often larger, and occasionally much larger. The figures, and the grouping, are the same ones published in [Chip Compatibility](#); the document is generated from the same source the CLI reads, so the two agree.

Chips are grouped by how they fit the board's socket, and the **Fit** column names the fit exactly:

Fit	Meaning
native	Chip and board have the same pin count — it goes straight in.
overhang	Chip has <i>fewer</i> pins than the board, so One ROM's top pins hang out of the socket.
larger socket (no fly-leads)	Chip has <i>more</i> pins than the board, but no address line among the extra ones: One ROM sits bottom-justified in the socket, with nothing to wire.
fly-lead to X1 / fly-lead to X1 and X2	Chip has more pins than the board, and the overhanging address line(s) must be wired to One ROM's X1 (and X2) header pin.

Every fit other than `native` is a cross-size fit, and in all of them One ROM's power pins may not line up with the socket's — **power must be rerouted to One ROM's own VCC/5V pin**. `larger socket (no fly-leads)` means no *signal* wiring is needed; it does not mean the chip simply drops in. Use `board socket` with `--chip-type` and `--gpio` to see exactly where One ROM's VCC lands.

The sizes are for a chip served alone in its slot. A banked or multi-ROM set draws further lines into the slot's address window, so its image can be larger than the figure shown here; build the firmware and run `onerom firmware inspect` to see what a specific set costs.

Chips are listed only where a size can be derived, which means Fire (RP2350) boards. An Ice (STM32) board falls back to a plain list of names. A chip type of the board's own pin count that the board cannot

serve — either because no firmware serves it yet (the SRAM types, at the time of writing) or because this particular board's layout cannot place it — is named in a trailing line rather than tabulated.

Board is taken from `--board`, or inferred from a connected One ROM. `--chip-type` accepts any chip type the board can emulate, under any accepted spelling.

Example output (illustrative — your build may differ):

```
$ onerom chips --board fire-24-f
Supported chip types for fire-24-f (One ROM Fire 24 (rev F)):
```

24-pin chips (native)				
Chip	ROM size	Image size	Fit	
2704	512B	512B	native	
2364	8KB	8KB	native	
...				

28-pin chips (with fly-leads)				
Chip	ROM size	Image size	Fit	
2764	8KB	32KB	fly-lead to X1	
...				

Image size is the flash One ROM uses to emulate the chip, which may exceed the chip's own ROM size. See docs/COMPATIBILITY.md.

Recognised but not servable on this board: 2016, 6116

Supported plugin types:
SystemPlugin, UserPlugin

A single chip type, on the board that makes the point — an 8KB ROM costing 256KB of flash, because One ROM overhangs a 28-pin socket to emulate a 24-pin part:

```
$ onerom chips --board fire-28-c --chip-type 2364
2364 on fire-28-c (One ROM Fire 28 (rev C)):
```

ROM size	8KB
Image size	256KB
Fit	overhang

With `--all`, chip types are listed by pin count without sizes, which are board-dependent:

```
Supported 24-pin chips:
 2016, 2316, 2332, 2364, 2704, 2708, 2716, 2732, 27C32, 28C16, 4732, 4764, ...
Supported 28-pin chips:
 231024, 23128, 23256, 23512, 23C1000, 23QL384, 23QL512, 27128, 27256, ...
Supported 32-pin chips:
 23C1001, 23C1010, 27C010, 27C020, 27C040, 29F010, 39SF010, SST39SF040, ...
Supported 40-pin chips:
 23C4100, 27C200, 27C400, 27C4100, AT27C400, HN62402, M27C400, MX23C4100, ...
```

Device required: no (a device is used only to infer the board when `--board` is omitted).

board

List supported One ROM board types, or draw a board's physical pin layouts as ASCII.

```
onerom board <COMMAND>
```

Subcommand	Purpose	Device required
list	List the supported board types	No
header	Draw a board's pin (jumper) header	No
socket	Draw a board's ROM socket pinout	No

board list

Lists the board types, in two groups. This replaces the bare `onerom boards` of earlier releases, which no longer exists.

The first group is the boards the CLI can act on — the Fire (RP2350) boards. The second is the Ice (STM32) boards, which the CLI recognises but cannot scan, program or build firmware for; naming one on a command that needs a device or an image is an error. Commands that only report hardware still take them.

```
onerom board list
```

Example output (illustrative — your build may differ):

```
Supported One ROM board types:
  fire-24-a, fire-24-c, fire-24-d, fire-24-e, fire-24-eadb01, fire-24-f, fire-24-g, fire-24-usb-b,
  fire-28-a, fire-28-b, fire-28-c, fire-28-d, fire-32-a, fire-32-b, fire-32-c, fire-40-a, fire-40-b,
  fire-40-c

Recognised, but not supported by the CLI:
  ice-24-d, ice-24-e, ice-24-f, ice-24-g, ice-24-i, ice-24-j, ice-24-usb-h, ice-28-a
  These boards use an STM32, rather than the RP2350 the CLI works with.
```

`onerom scan --list-boards` prints the same listing.

Device required: no.

board header

Draw a board's pin (jumper / programming) header — the 2xN header along the board's top edge — as ASCII, pad by pad. Each image-select and X pad is annotated with the MCU GPIO behind it, and on RP2350 (Fire) boards with whether that GPIO is 5V-tolerant (5V) or 3.3V-only (!3V3!! — an ADC pin that must not be driven above 3.3V). See [Voltage Levels](#) for the ADC caveat.

```
onerom board header [--board <board>]
```

Option	Description
<code>--board, -b</code>	Board type to draw (e.g. fire-24-f). Inferred from a connected One ROM if omitted.

A board with no pin-header descriptor prints a short notice instead of a diagram.

```
onerom board header --board fire-24-f
```

With no `--board`, the CLI takes the board type from the connected One ROM. It cannot do that for a device whose firmware it cannot read, and says so:

```
$ onerom board header --unrecognised
Failed to execute command.
Could not determine board type from the connected device Unknown      - Firmware: n/a   State:
Unknown Serial: (no serial).
  It may be an unprogrammed One ROM or have corrupt firmware.
  Supply the board type with --board
```

The header carries the `B00TSEL` pad used to boot a One ROM into its own bootloader — see [Recovering a bricked One ROM](#).

Device required: no (a device is used only to infer `--board` when it is omitted).

board socket

Draw a board's ROM socket pinout as a DIP diagram.

```
onerom board socket [--board <board>] [--chip-type <chip>] [--gpio]
```

Option	Description
<code>--board, -b</code>	Board type to draw (e.g. fire-24-f). Inferred from a connected One ROM if omitted.
<code>--chip-type, -c</code>	Label pins with this chip type's ROM functions instead of GPIOs.
<code>--gpio</code>	Overlay the GPIO(s) behind each pin onto the <code>--chip-type</code> view. Requires <code>--chip-type</code> .

Without `--chip-type`, each socket pin is labelled with the GPIO(s) behind it (the GPIO map). With `--chip-type <chip>` (e.g. 2364), the pins are labelled with that ROM's functions (address / data / chip-select / BYTE / power / ...) instead; add `--gpio` to overlay both. `--gpio` requires `--chip-type`. A pin that carries two functions on a multiplexed part (e.g. the 27C400's pin 29, A0/D15) shows both.

The two views that label pins with GPIOs — no `--chip-type`, or `--gpio` — need the board's GPIO map. A board without one reports that and draws nothing; the `--chip-type` function view still works, as it is drawn from the chip's pinout and the board's ROM signal assignments.

When the chip's pin count differs from the board's, the socket is drawn at the larger of the two and the smaller device is bottom-justified (see [Chip Compatibility](#)):

- emulating a **smaller** ROM on a larger One ROM, One ROM's extra pins hang out of the socket and are marked `overhang` (reroute power to One ROM's VCC/5V pin);

- emulating a **larger** ROM on a smaller One ROM, the socket pins One ROM does not reach are marked (empty) , and any address line there shows the One ROM X1 / X2 header pin it must be fly-leaded to (e.g. A12 → X1).

In both cases One ROM's own power pins may not line up with the ROM's. With `--gpio` , the pin One ROM's VCC (or GND) lands on is annotated (VCC) / (GND) — e.g. NC (VCC) shows One ROM's VCC sitting on the ROM's NC pin — so you know where power must be applied.

With `--chip-type` , the diagram is followed by that chip's image size on this board — the flash One ROM uses to emulate it, as reported by `onerom chips` .

`--chip-type` must be a chip type the board can emulate (native, overhang or fly-lead; see `onerom chips` and [Chip Compatibility](#)).

```
onerom board socket --board fire-24-f
onerom board socket --board fire-24-f --chip-type 2364
onerom board socket --board fire-24-f --chip-type 2364 --gpio
```

Device required: no (a device is used only to infer `--board` when it is omitted).

self

Check for, and download, new releases of the CLI itself. This is the CLI's own release channel — the binaries published at <https://onerom.org/cli> — and is separate from the One ROM firmware releases `firmware releases` lists.

Nothing here runs unless you ask for it: the CLI performs no update check of its own accord, and neither command installs anything.

```
onerom self check
onerom self download
```

Device required: no.

self check

Compare this build against the newest release published for this platform.

```
onerom self check
```

This command takes no options.

It prints the running version, then one of three things: that this is the latest release; that a newer one is available, with how to get it; or — for a build made from source — that this build is newer than anything published.

Finding an update is not an error: the exit code is 0 in all three cases. A non-zero exit means the check itself failed, such as an unreachable images server, so a script can tell "no update" from "could not tell".

self download

Download a published CLI release.

```
onerom self download
onerom self download --version 0.3.0 --path ~/Downloads
onerom self download --target aarch64-unknown-linux-gnu
onerom self download --target all --path ./dist
```

Option	Description
<code>--version <VERSION></code>	Version to download (e.g. 0.3.0). Defaults to the latest release.
<code>--target <TARGET></code>	Platform to download for, as a target triple. Defaults to this machine's. all downloads every platform's artifact for the version, and requires <code>--path</code> .
<code>--output, -o <FILE></code> (alias <code>--out</code>)	Output file path. Defaults to the published filename. Conflicts with <code>--path</code> .
<code>--path <DIR></code>	Output directory, using the published filename. Conflicts with <code>--output</code> .

Option	Description
<code>--force, -f</code>	Overwrite an existing file.

The downloaded file is checked against the SHA-256 published alongside it; a mismatch is reported and the download discarded. That digest comes from the same server as the file, so it catches a corrupted or truncated download rather than a compromised server — it is not a signature. The Windows and macOS builds are digitally signed, and that signature is what your OS checks when you run them.

The published filename already carries the version and architecture, so it is the default name in both the current directory and a `--path` directory. Every output path is checked before anything is downloaded, so an existing file or a missing directory fails immediately rather than part-way through a `--target all` run.

Platform names are the Rust target triples the manifest publishes: `x86_64-unknown-linux-gnu`, `aarch64-unknown-linux-gnu`, `x86_64-pc-windows-msvc`, `aarch64-pc-windows-msvc`, and `universal-apple-darwin`. The last is not a real triple: the macOS build is a universal binary covering both Apple architectures. Naming an unknown one lists what is published.

`--target` is there to fetch a build for a machine other than this one — the ARM `.deb` for a Raspberry Pi, say, from a desktop. When the artifact is not for this platform, no install step is printed, since the local one would not apply.

Nothing is installed and the running `onerom` is not replaced. Install what was downloaded the same way you would install it from the website — see [Installation](#).

Top-level aliases

Convenience aliases for frequently used nested commands. They take the same options as their targets.

peek (top-level alias)

Alias for `inspect peek live`.

```
onerom peek --address 0x100 --length 64
```

poke (top-level alias)

Alias for `control poke live`.

```
onerom poke --address 0x100 --input patch.bin
```

reboot (top-level alias)

Alias for `control reboot`.

```
onerom reboot
```

ROM slot specification

Used by `--slot` in `program` and `firmware build`. Repeat `--slot` once per slot. Comma-separated key=value pairs:

```
file=<path_or_url>,type=<romtype>[,label=<text>]
[,cs1=<logic>][,cs2=<logic>][,cs3=<logic>][,cs4=<logic>]
[,size-handling=<handling>][,format=<binary|ihex|srec>][,load-address=<addr>]
[,transform=<list>]
[,cpu-freq=<freq>][,cpu-vreg=<voltage>][,led=<bool>][,force-16-bit=<bool>]
```

Key	Values / notes
file	Local path or URL to the ROM image.
label (alias name)	A name for this image, recorded in the device metadata in place of the filename, and shown by <code>scan --slots</code> , <code>inspect slots</code> and <code>firmware inspect</code> . Worth setting when the file is a long path or URL, or when the recorded name would otherwise be truncated — see Image transforms . The same field is label in a config file.
type	Chip type, e.g. 2364, 2332, 2716, 27C400. Any type the target firmware can serve on the board is accepted — that is exactly what <code>chips --board</code> lists, including the overhang and fly-lead combinations (a 2764 on a Fire 24, say); see COMPATIBILITY.md . Building for firmware older than v0.7.0 accepts a narrower set, and a rejection lists what that firmware serves. Any accepted alias may be used; the exact spelling you enter is preserved in the device metadata (shown by <code>scan/inspect</code>), while the resolved type drives behaviour.
cs1, cs2, cs3, cs4	CS polarity: active-low (or 0), active-high (or 1), or ignore. The snake_case config spellings (active_low, active_high) are also accepted. Which lines are required depends on the chip type (e.g. 2332 requires cs1 and cs2). A chip type without that line, or with its polarity fixed in silicon, rejects it — CHIP-TYPES.md lists each type's control lines. ignore says One ROM does not monitor the line at all — it is not a polarity, and is only permitted where the chip type or set allows it (see allow_cs_ignore).
size-handling (aliases size, size_handling)	none, duplicate (or dup), truncate (or trunc), pad. For an Intel HEX or S-record image, padding fills with 0xFF and duplicate is not permitted.
format	binary (default), ihex (Intel HEX) or srec (Motorola S-record). An ihex or srec file is decoded to a binary image before use; unwritten bytes read as 0xFF.
load-address (alias load_address)	Only valid with format=ihex or format=srec. The absolute address that maps to byte 0 of the ROM, as a decimal or 0x/\$-prefixed hex value (e.g. \$E000). Defaults to 0.
transform	Byte-level rearrangements of the image, applied in the order given and joined with +. See Image transforms .
cpu-freq	e.g. 150, 150mhz, 150MHz. Values above 150 MHz require confirmation (suppressed by --yes) and set overclock automatically.
cpu-vreg	e.g. 1.1, 1.10, 1.10v, 1.10V. Values above 1.10 V require confirmation (suppressed by --yes). Must be a supported level.

Key	Values / notes
led	Boolean: on/off, true/false, 1/0.
force-16-bit (alias force_16bit)	Boolean (as above). Valid only on 40-pin boards.

Examples:

```
--slot file=kernal.bin,type=2364,cs1=active-low
--slot file=chargen.bin,type=2332,cs1=active-low,cs2=active-high
--slot file=https://example.com/basic.bin,type=2716
--slot file=https://example.com/c64/roms/901227-03.bin,type=2364,cs1=active-low,label=kernal
--slot file=small.bin,type=2364,cs1=active-low,size-handling=duplicate
--slot file=kernal.hex,type=2364,cs1=active-low,format=ihex
--slot file=kernal.hex,type=2364,cs1=active-low,format=ihex,load-address=$E000
--slot file=kernal.s19,type=2364,cs1=active-low,format=srec,load-address=$E000
--slot file=kernal.bin,type=2364,cs1=active-low,cpu-freq=200MHz,cpu-vreg=1.2V
--slot file=char.bin,type=2332,cs1=active-low,cs2=active-high,led=off
--slot file=amiga.bin,type=27C400,force-16-bit=true
--slot file=undersized.bin,type=2732,size=pad
--slot file=oversized.bin,type=2732,size=trunc
--slot file=halfsized.bin,type=2732,size=dup
--slot file=amiga.bin,type=27C400,transform=swap_bytes
--slot file=rom32.bin,type=27C010,transform=deinterleave:1/2/2+swap_bytes
```

Image transforms

Some ROM images are not laid out the way the target chip needs them: a 16-bit part whose image was produced with the opposite byte order, or a wide image that interleaves several narrower devices. Transforms rearrange the bytes before the image is written into the firmware.

They are available two ways, and both run exactly the same operation:

- as the `transform=` key of a `--slot`, or a "transform" array in a config file, applied during the build;
- as the standalone `image swap-bytes` and `image deinterleave` subcommands, which rewrite a file.

Transform	Effect
<code>swap_bytes</code>	Reverses the byte order within each 16-bit word. The image must have an even length.
<code>deinterleave:<offset>/<stride></code>	The image contains <code>stride</code> interleaved lanes one byte wide; keep lane offset.
<code>deinterleave:<offset>/<stride>/<bytes></code>	As above, with lanes bytes wide — use 2 to keep 16-bit words together.

The parameters are positional, in the order `<offset>/<stride>/<bytes>`, and `<bytes>` may be omitted (it defaults to `1`).

Each name has aliases, accepted identically by the CLI and by a config file:

Canonical	Also accepted
<code>swap_bytes</code>	<code>swap-bytes</code> , <code>swapbytes</code>
<code>deinterleave</code>	<code>de_interleave</code> , <code>de-interleave</code> , <code>deint</code>
<code>transform=</code> (slot key)	<code>trans=</code>

`deinterleave` requires the image length to be a multiple of `bytes × stride` — one full set of lanes. The result is `1/stride` of the input length.

Transforms are applied **in the order listed**, and the order matters:

```
transform=deinterleave:1/2/2+swap_bytes
```

takes the upper 16-bit half of each 32-bit word and *then* swaps its byte pairs. Note that `offset` selects which lane, not a named "high" or "low" half — which half you get depends on the byte order of your source image, and that is what `swap_bytes` is for.

Within the build pipeline, transforms run after any `location` window and after an Intel HEX or S-record image has been decoded, but before `size-handling` reconciles the image against the chip size. A `swap_bytes` on an odd-length image is an error unless `size-handling` is `pad` (which appends one blank byte) or `truncate` (which drops the trailing byte). Where the size handling is used this way it counts as having been needed, so it is not then reported as redundant even if the transformed image lands on exactly the chip size.

Common recipes:

Goal	Spec
16-bit image with the wrong byte order	<code>transform=swap_bytes</code>
Even / odd bytes of a 16-bit interleaved image	<code>transform=deinterleave:0/2</code> / <code>transform=deinterleave:1/2</code>
Byte <i>n</i> of a 32-bit interleaved image	<code>transform=deinterleave:<n>/4</code>
One 16-bit half of a 32-bit interleaved image	<code>transform=deinterleave:0/2/2</code> or <code>deinterleave:1/2/2</code>

The transforms applied to an image are recorded in the firmware metadata alongside its filename — as `kick.bin|transform=swap_bytes` — so a built image carries a record of how its ROM data was derived. Note that the metadata filename field is capped at 128 bytes, so the suffix can be truncated away for a very long path; use `label=` to keep it short.

16-bit ROM image byte ordering

A 16-bit ROM supplies two bytes at a time, and an image of one may hold each pair in either order. One ROM reads the low byte of each pair first. An image holding the high byte first needs `swap_bytes`, and without it every pair is served reversed and the machine does not work.

The CLI checks for this. It compares the first bytes of the image against a list of known ROM headers, including ones seen in Amiga Kickstart, DiagROM and Atari ST TOS images. An image matching no entry is left alone.

A 16-bit image holding the high byte of each pair first, with no transform:

```
$ onerom firmware build --board fire-40-a --slot file=kick.bin,type=27C400 --out fw.bin
Warning: kick.bin starts with an Amiga ROM header, high byte of each pair first.
One ROM needs the low byte of each pair first. Add transform=swap_bytes to
this slot.
```

`swap_bytes` applied to an image that did not need it:

```
$ onerom firmware build --board fire-40-a --slot file=kick-swapped.bin,type=27C400,transform=swap_bytes --out fw.bin
Warning: kick-swapped.bin starts with an Amiga ROM header and was already the way One
ROM needs it. transform=swap_bytes has swapped it the wrong way round.
Remove transform=swap_bytes from this slot.
```

Neither is refused. The build and the programming go ahead.

The check is skipped for an 8-bit chip, whose image holds no 16-bit words, and for a slot carrying any transform besides `swap_bytes`, since the check then reads bytes that are not the ones served.

`--verbose` also reports a recognised image that needs no change, and one the check could not identify:

```
$ onerom --verbose image swap-bytes --input kick.bin --output out.bin
kick.bin starts with an Amiga ROM header, high byte of each pair first.
Swapping the bytes makes it correct for One ROM.
```

```
$ onerom --verbose firmware build --board fire-40-a --slot file=blank.bin,type=27C400 --out fw.bin
Unable to tell which way around the byte pairs are in blank.bin. If the slot
does not work, try transform=swap_bytes.
```

Plugin specification

Used by `--plugin` in `program` and `firmware build`. At most one system plugin and one user plugin; a user plugin requires a system plugin. The system plugin is placed in slot 0, the user plugin in slot 1.

Form	Meaning
<code>--plugin usb</code>	Latest compatible version, by name.
<code>--plugin system/usb</code>	With explicit type (system or user).
<code>--plugin usb,version=0.1.0</code>	Pinned version.
<code>--plugin file=path/to/plugin.bin</code>	Local file.
<code>--plugin file=https://example.com/plugin.bin</code>	Remote file.

Named forms are selected against the release manifest, so an incompatible one is refused at that point. A `file=` form, and a plugin named by a config, are checked separately — see [Plugin compatibility](#).

Pin values

Used by `--pin` in `control pin`, `control reset` and `inspect gpio`, and by `--reset-host` in `program`.

`--pin` names one **MCU GPIO**, either directly or through a header pad that is wired to one. All spellings are case-insensitive (`GPI023` , `SEL_A`).

Form	Meaning
<code>gpio<N></code>	An MCU GPIO — for example <code>gpio23</code> .
<code>sel_a ... sel_e</code>	An image-select pad. <code>sel-a</code> and <code>sela</code> are also accepted.
<code>x1, x2</code>	An X pad.

A pad name resolves against the **board**, since which GPIO sits behind `sel_a` is a fact about the board and not about the name. The board is normally read from the connected device; `--board` overrides it, and is what you need if this build does not recognise the device's board revision. `gpio<N>` needs no board. A board that has no such pad — `sel_e` on a four-select board, `x1` on a board with no X pads — is an error naming the pads that board does have.

Resolution uses the board's electrical pin assignments, not its header layout, so pad names work on every board, including those whose physical header is not yet characterised.

A bare number is **rejected**. `23` could be an MCU GPIO, an image-select pad, an X pad or a ROM socket pin, and driving the wrong one is not a recoverable mistake, so the CLI names the namespaces rather than guessing. Accepting pad names does not remove that ambiguity — it sharpens it.

The broken-out address pads (`a<N>`) are recognised and **deliberately refused**, now and in future. `--pin` addresses MCU GPIOs and the pads a wire can reach; an address line is a ROM signal rather than one of those, and accepting `a17` would invite `a11` or `d3`, which have no pad at all. Use the MCU GPIO behind the pad. `run`, `bootsel`, `swclk` and `swdio` are reported as not being GPIOs that can be driven. There is no syntax for a ROM socket leg.

Run `onerom inspect header` to see which GPIO is behind each header pad, or `onerom inspect gpio` for the full per-GPIO listing.

The upper bound is the device's own GPIO count — 30 on an RP2350A, 48 on an RP2350B — read from the device rather than assumed, so a GPIO the device does not have is reported against what it does have.

Problems

Recovering a bricked One ROM

You can recover a One ROM that is not responding using any of the One ROM programming tools by following the instructions below. The One ROM CLI is recommended as it gives greatest control over One ROM. The CLI commands are shown below.

Situation

No tool can find the device. `onerom scan` reports nothing, the browser programmer sees nothing to connect to, and any command needing a device refuses. The Web programmer cannot detect it and the CLI reports:

```
$ onerom scan
Scanning ...
No matching One ROM devices found.

$ onerom inspect info
Failed to execute command.
No One ROM was found or specified.
Specify a One ROM using --serial.
Use 'onerom scan' to list connected One ROMs.
```

A One ROM in this state is called bricked. Nothing is damaged. Its firmware is not running, so nothing answers on the USB bus — programming was interrupted, or the firmware on it is not right for the board. One ROM has a hardware bootloader which cannot be bricked, so the recovery is to boot the device into that bootloader and program it again.

If the One ROM programming tool you are using does find the device, it is not bricked. Program it as normal.

Booting into the bootloader

This works on any Fire (RP2350) board whatever state its flash is in.

1. Unplug the One ROM.
2. Connect the **BOOTSEL** pad to ground. It is normally the middle pad/pin on the header pins' top row, and the USB shield is a good source of ground. The CLI command `onerom board header --board <BOARD>` shows the header pins.

Fire 24 rev A and Fire 24 USB rev B are the exceptions, and both are rare. Rev A brings BOOTSEL out as a pin towards the bottom of the board, and USB rev B as a small pad on the underside.

3. Plug the One ROM into USB with that connection still made. The status LED lights dimly, which is how you know the bootloader is running.
4. Remove the BOOTSEL to ground connection — it is needed only as power comes up.

Checking the host can see it

Connect to One ROM using the programming tool as normal.

Using the CLI `--unrecognised` (`-u`) matches any attached RP2350 board, including a Raspberry Pi Pico 2, so make sure only the One ROM is attached:

```
$ onerom scan --unrecognised
Scanning ...
found 1 connected device:
  Unknown          - Firmware: n/a   State: Unknown Serial: (no serial)
```

The CLI names a device's board, firmware and serial from the firmware it is holding, and there is none in this example's bricked One ROM it can read.

Programming it again

You have to supply the One ROM board information. A One ROM board type is only identifiable from the firmware already programmed to it, and that is the thing that is missing or wrong. The board name is the pin count and the revision letter silkscreened on the board — `fire-24-f` is a 24-pin board, revision F. The marking is small. `onerom board list` prints every name.

To re-program with the CLI, add `--unrecognised` and `--board`:

```
onerom program --unrecognised --board fire-24-f --config c64.json
```

With the [browser programmer](#), pick the board yourself in the same way. It will ask you to confirm the board type before it writes.

If the board was mis-flashed rather than left blank, both tools notice — the wrong firmware is still in flash and they read the board from it. The CLI refuses, and needs `--force` alongside `--board`. The browser programmer warns and lets you continue. Check the silkscreen once more before you do either, because the same objection appears when the board is right and the name you picked is wrong.

Then confirm the device came back up. With the CLI:

```
onerom scan
```

Getting the board wrong writes the wrong firmware and leaves you with a bricked device. In this case, follow the instructions again.

Ice boards

Ice (STM32) boards use `B00T0` rather than `BOOTSEL`, and it is pulled **high**, to 3.3V, rather than to ground. It is the jumper labelled `B0` or `B`. The 0.7.x CLI does not program Ice boards at all — use the [Web Programmer](#) or [One ROM Studio](#).

Appendix: Breaking Change History

Changes that can alter or break a command line that worked on an earlier release. Newest release first.

v0.4.0

- `--name` is an alias for `--instance-name` on `program` and `firmware build`. It was an alias for `--config-name`. A command line using `--name` alongside `--slot` still runs, and names the One ROM rather than the configuration. With `--no-config` it is now rejected.

v0.3.0

- `onerom boards` is now `onerom board`, and the bare listing it printed is `onerom board list`. There is no alias.
- `onerom control erase` takes `--stopped` / `--running` for its post-erase reboot mode, in place of `--reboot-stopped` / `--reboot-running`. There is no alias.
- `--allow-unsupported-chip-type` is removed from `program` and `firmware build`. Every chip type the target firmware can serve is accepted without it.
- No command takes a positional argument. `board header` and `board socket` take `--board`.
- Each short flag means one thing across the whole CLI: `-b` is `--board` (was `--byte` on `poke`), `-o` is `--output` (was `--offset` on `control erase`), `-i` is `--input` (was the global `--vid-pid`, which keeps `--id`), `-l` is `--length` (was `--slot`), and `-m` is `--msd` (was `--image` on `update slot`).
- `firmware build` no longer accepts `--swd_disabled`. `--disable-swd` and `--swd-disable` work on both it and `program`.